

1 fMRI for Program Comprehension

Motivation: Traditional studies rely on indirect measures (performance, surveys, think-aloud) that fail to capture actual cognitive processes in program comprehension. There is a need for objective, physiological evidence of how developers understand code. Methodology: fMRI is used to observe brain activity while participants perform controlled comprehension tasks, with baseline comparisons to isolate code-specific reasoning. Key Findings: Code comprehension activates left-hemisphere regions associated with working memory, attention, and language, indicating a hybrid cognitive activity distinct from pure natural language or mathematics. Different code structures produce varying cognitive loads. Implications: Enables objective measurement of cognitive effort, allowing comparison of programming languages, tools, expertise levels, and code complexity to design better developer environments and education.

2 Code Readability and Cognitive Load

Motivation: Code readability and naming are believed to affect comprehension, but their cognitive impact is not well quantified. Methodology: Controlled experiments manipulate identifier quality and structural readability while measuring task performance and physiological cognitive load. Key Findings: Poor identifiers and inconsistent naming significantly increase cognitive load; structural complexity further amplifies difficulty, and combined effects degrade comprehension accuracy and speed. Developers spend more effort decoding poorly written code. Implications: Readability is a fundamental quality attribute; enforcing naming conventions, modular design, and clean structure reduces mental effort and improves productivity and maintainability.

3 Code Review and Expertise in the Brain

Motivation: It is unclear whether code is processed like natural language or as a distinct cognitive task, and how expertise changes this process. Methodology: fMRI experiments compare brain activity across code comprehension, code review, and prose reading tasks, using machine learning classifiers to distinguish neural patterns. Key Findings: Code tasks produce distinct neural signatures compared to prose, but experienced developers show greater overlap, suggesting abstraction and familiarity. Expertise reduces cognitive effort and enables more efficient processing. Implications: Programming is a unique cognitive domain; expertise reshapes mental models, which has implications for training, onboarding, and designing developer tools that support different skill levels.

4 Learning a New Programming Language

Motivation: Developers often struggle to learn new languages despite prior experience due to incorrect transfer of knowledge. Methodology: Mixed-method study combining Stack Overflow analysis with interviews to understand learning behaviors and challenges. Key Findings: Cross-language interference is common, where developers apply incorrect assumptions from known languages; just-in-time learning strategies are efficient but lead to shallow understanding and fragile mental models. Misconceptions persist without explicit correction. Implications: Learning resources should emphasize conceptual differences and transitions between languages, helping developers rebuild mental models rather than simply learning syntax or APIs.

5 Predictors of Developer Productivity

Motivation: Developer productivity is difficult to define and measure, and its key drivers are unclear. Methodology: Large-scale survey with regression analysis examining technical, social, and organizational factors affecting self-reported productivity. Key Findings: Social and psychological factors (motivation, peer support, feedback, autonomy) are stronger predictors than purely technical aspects; interruptions and work environment significantly reduce productivity. Individual skill alone is insufficient to explain differences. Implications: Improving productivity requires organizational changes, better communication, and supportive environments, not just better tools or increased individual effort.

6 Developer Productivity and Code Quality at Google

Motivation: Prior studies show correlations but lack causal evidence linking productivity and engineering practices. Methodology: Longitudinal analysis combining survey responses and engineering logs using fixed-effects and lagged regression models to infer causality. Key Findings: Code quality is the strongest predictor of future productivity; improvements in code health precede measurable gains in efficiency; stable priorities and tool satisfaction also contribute. Technical debt negatively impacts productivity over time. Implications: Investing in code quality, refactoring, and maintainability directly improves short-term and long-term productivity.

7 Logs-Based Focus and Flow

Motivation: Flow is critical for productivity but difficult to measure without intrusive self-reporting. Methodology: Multi-phase study using developer activity logs, embeddings of actions, and validation against surveys to define behavioral metrics of focus. Key Findings: Flow cannot be directly inferred from logs, but focused work can be detected behaviorally; frequency of focus sessions correlates more strongly with productivity than duration alone. Interruptions disrupt focus patterns. Implications: Provides scalable, passive measurement of focus, enabling organizations to analyze work patterns and design environments that reduce interruptions and improve sustained productivity.

8 GenAI, Creativity, and Software Development

Motivation: Research on generative AI focuses on productivity, while its effects on creativity are less understood. Methodology: Conceptual analysis using the 4P creativity framework (Person, Process, Product, Press) and McLuhan's tetrad to explore potential impacts. Key Findings: AI enhances ideation, accelerates exploration, and reduces routine effort, but may lead to homogenization of solutions, reduced originality, and over-reliance on generated outputs. Creative processes may shift from generation to evaluation. Implications: AI should augment human creativity rather than replace it; research must ensure diversity, innovation, and critical thinking are preserved in AI-assisted development.

9 Beliefs, Practices, and Personalities of Engineers

Motivation: The relationship between personality traits, engineering practices, and beliefs is not well understood in large-scale settings. Methodology: Industrial survey using the Five-Factor Model along with questions on practices, tools, and preferences. Key Findings: Strong agreement on practices such as testing, readability, and code reviews; disagreement on tools, workflows, and environments; personality differences across roles are minimal, with slight variation for managers. Shared practices dominate over individual traits. Implications: Engineering effectiveness is shaped more by culture and practices than personality; teams should focus on establishing strong shared norms and processes.

10 Requirements Elicitation Survey

Motivation: Requirements elicitation is a foundational but difficult phase because stakeholder needs are often ambiguous, incomplete, distributed, and evolving. It is not just a matter of collecting requirements, but of discovering, clarifying, and sometimes even inventing them through communication and analysis. Methodology: The paper surveys elicitation techniques, approaches, and tools, organizing them around activities such as understanding the domain, identifying stakeholders, selecting techniques, and refining requirements. It covers interviews, questionnaires, task analysis, ethnography, prototyping, goal-based approaches, scenarios, and viewpoints. Key Findings: No single technique works best in all situations; interviews and workshops provide rich stakeholder knowledge, ethnography and observation reveal tacit practices, and prototyping helps stakeholders articulate needs more concretely. Effective elicitation is inherently iterative, multidisciplinary, and context dependent. Implications: Analysts must combine techniques strategically based on domain complexity, stakeholder availability, and organizational constraints; elicitation should be treated as both a technical and social process requiring facilitation, negotiation, and continuous validation.

11 Techniques and Approaches in RE

Motivation: Analysts often struggle to choose elicitation methods because each technique has different strengths, costs, and assumptions. Methodology: Comparative discussion of techniques across elicitation activities such as domain understanding, stakeholder analysis, requirement discovery, and validation. The survey contrasts generic techniques like interviews and group work with structured approaches such as goal modeling, scenarios, and viewpoints. Key Findings: Interviews are flexible and widely applicable but depend heavily on analyst skill; questionnaires scale well but provide limited depth; observation and ethnography capture real behavior but require time and access; prototyping improves clarity but may prematurely anchor stakeholder expectations. Goal-based, scenario-based, and viewpoint-oriented techniques provide stronger structure and traceability, especially in complex systems. Implications: Technique choice should reflect elicitation purpose rather than habit or familiarity; combining complementary methods increases completeness and reduces blind spots, while substitute techniques can provide flexibility when ideal methods are infeasible.

12 Methodology-Based RE

Motivation: Elicitation is often not performed as a standalone activity but is embedded within larger development methodologies. Methodology: The chapter reviews methodology-based elicitation through Structured Analysis and Design, object-oriented approaches and UML, integrated exploratory methods, quality-driven methods such as QFD, and Agile practices. Key Findings: Structured approaches help formalize requirements through artifacts like DFDs, ERDs, and Use Cases; exploratory and soft-systems methods foreground organizational context and stakeholder worldviews; Agile shifts elicitation from upfront specification to continuous discovery through user stories and evolving backlogs. Different methodologies emphasize different balances between rigor, flexibility, and stakeholder collaboration. Implications: Elicitation should align with the overall process model being used; analysts need to understand how methodology shapes what gets captured, when it gets captured, and how much stakeholder feedback can be incorporated over time.

13 Tool Support in Requirements Engineering

Motivation: Manual RE processes are difficult to scale, especially when many stakeholders, artifacts, and changes must be managed over time. Methodology: Survey of tools ranging from lightweight templates and structured specifications to requirements management systems, modeling tools, and collaborative groupware. Key Findings: Tools help document requirements consistently, maintain traceability, and support distributed collaboration, but they do not remove the need for analyst judgment. Advanced tools for goal modeling, scenarios, and automated assistance

exist, yet industrial adoption remains uneven because usability, integration, and practical fit matter as much as functionality. Implications: RE tool support is most effective when it augments human communication and analysis rather than trying to replace them; future tooling should better support technique selection, contextual adaptation, and AI-assisted but transparent requirement refinement.

14 Issues and Pitfalls in RE

Motivation: Requirements engineering failures remain a major source of project cost, delay, and system mismatch, despite decades of methodological research. Methodology: The chapter synthesizes recurring pitfalls reported in both research and practice, including process-related, communication-related, stakeholder-related, and analyst-related problems. Key Findings: Common issues include unclear project scope, stakeholder conflicts, incomplete and inconsistent requirements, communication breakdowns, volatility of expectations, and premature fixation on solutions instead of problems. Analysts may also default to familiar techniques instead of selecting methods appropriate to context, while organizations often underinvest in elicitation effort. Implications: Successful elicitation requires iterative clarification, sustained stakeholder engagement, and explicit attention to ambiguity, conflict, and change; quality problems in requirements are rarely just documentation issues and usually reflect deeper communication and process failures.

15 Trends and Challenges in RE

Motivation: Modern software projects involve increasing scale, distribution, volatility, and socio-technical complexity, making traditional elicitation practices harder to sustain. Methodology: Survey of emerging research and practice trends, including formalization, contextual adaptation, Agile integration, distributed collaboration, and tool-supported elicitation. Key Findings: Research is moving toward structured model-driven approaches and better support for agile, web-based, distributed, and agent-oriented development. In practice, interviews and group meetings remain dominant, while Use Cases, UML, and lightweight agile artifacts are widely adopted. Major open challenges include weak empirical validation of many techniques, lack of standardized evaluation metrics, and the persistent gap between proposed methods and what industry actually uses. Implications: RE must become more adaptive, evidence-based, and better integrated with real workflows; researchers need stronger empirical grounding, and practitioners need simpler, more actionable guidance for selecting and combining techniques.

16 Conversational AI in RE Education

Motivation: Conversational AI tools such as ChatGPT are rapidly entering software engineering education, but their effect on RE learning and student reasoning is not yet well understood. Methodology: Course-based empirical study in which student groups used ChatGPT during requirements specification under two different workflows: human-first drafting followed by refinement, and AI-first generation from the start. Final artifacts, intermediate drafts, and interaction logs were analyzed. Key Findings: Both workflows produced similar average artifact quality, but high-quality results still required substantial effort in prompting, refinement, and critical evaluation. Students mainly used AI for ideation, rewriting, and concretization of existing ideas. Prompt quality strongly influenced output quality, while weak prompts often led to generic, shallow, or misaligned requirements. Implications: AI can support RE learning, but only when paired with explicit instruction in prompt design, critical reading, and validation; educational use should emphasize collaboration with AI rather than passive acceptance of outputs.

17 Prompting and Student Interaction with AI

Motivation: The usefulness of conversational AI depends not just on the model itself, but on how users interact with it. Methodology: Analysis of full student interaction logs to study prompting behavior, prompting quality, and effort patterns across RE tasks. Key Findings: Students often lacked prompting proficiency, failing to give enough context, constraints, or structure. Many were overly influenced by the confidence of ChatGPT responses and sometimes accepted flawed or overly generic requirements without sufficient critique. The best outcomes occurred when students treated prompting as an iterative design activity rather than a one-shot command. Implications: Prompt engineering and output evaluation are emerging core skills in AI-assisted software work; teaching students how to question, refine, and verify AI output is as important as teaching them how to generate it.

18 Monolithic Repository Advantages and Disadvantages

Motivation: Large software organizations must decide whether to organize code in a monolithic repository or across many repositories, but empirical evidence on trade-offs is limited. Methodology: Mixed-method case study at Google combining a large-scale survey of engineers with developer tool log analysis to compare perceptions and actual behavior around repository visibility, dependency management, and cross-project work. Key Findings: Developers generally prefer monolithic repositories because global visibility helps them discover reusable APIs, inspect example usage, and coordinate changes across dependencies. Centralized tooling and synchronized dependency updates are major strengths. However, multi-repository systems provide more autonomy, flexible toolchains, and clearer version boundaries, and may reduce perceived complexity for smaller teams. Implications: Repository choice is a socio-technical trade-off between visibility and standardization on one side, and flexibility and local autonomy on the other; organizations should evaluate repository

strategy in light of tooling, dependency coordination, and developer workflows.

19 Diversity in Software Engineering Teams

Motivation: Building and sustaining diverse software engineering teams is important for equity, inclusion, and organizational effectiveness, yet the mechanisms that support diversity are not always visible in practice. Methodology: Industrial qualitative study examining how software engineering teams at Google recruit, retain, and support ethnically, racially, and gender diverse members. The study focuses on practices, organizational mechanisms, and lived experiences rather than only demographic counts. Key Findings: Diversity work depends on intentional recruiting practices, allyship, inclusive team norms, and active management support rather than passive belief in meritocracy alone. Technical allyship, advocacy, and structurally supported inclusion all play important roles in sustaining team diversity. Implications: Diversity should be treated as an ongoing organizational practice rather than a one-time hiring goal; managers and engineers both need concrete behaviors and structures that support inclusion, participation, and long-term retention.

20 Costs and Benefits of Pair Programming

Motivation: Pair programming is widely discussed as a collaborative development practice, but its trade-offs across productivity, quality, learning, and team dynamics must be understood in detail. Methodology: Empirical synthesis of the reported advantages and disadvantages of pair programming across different dimensions of software work. Key Findings: Pair programming can improve design quality, knowledge sharing, team communication, learning, and defect detection, especially in complex or uncertain tasks. At the same time, it can increase immediate effort, require interpersonal compatibility, and feel inefficient for routine or individual work. The balance of cost and benefit depends strongly on task type and team context. Implications: Pair programming should not be treated as universally beneficial or universally wasteful; it works best when used deliberately for learning, design-heavy tasks, and coordination-intensive work rather than as a blanket policy.

21 Shared Gaze in Remote Pair Programming

Motivation: Remote pair programming can suffer from reduced awareness and weaker communication because collaborators cannot easily see what the other person is attending to. Methodology: Study of shared gaze awareness as a mechanism for improving communication and coordination between pair programmers. Key Findings: Making gaze and attention more visible can improve grounding, reduce misunderstanding, and help partners coordinate discussion around code artifacts more effectively. However, the benefits depend on how naturally the awareness mechanism integrates into the workflow. Implications: Remote collaboration tools should support richer awareness cues, not just screen sharing or text chat, because shared attention is a meaningful part of collaborative programming quality.

22 Modern Code Review

Motivation: Code review is central to software quality and team coordination, but its actual goals, outcomes, and challenges extend beyond simple defect finding. Methodology: Empirical study of expectations, outcomes, and challenges in modern code review, focusing on how practitioners use review in real settings. Key Findings: Code review supports defect detection, knowledge transfer, maintainability improvement, and social coordination, but also faces challenges such as reviewer overload, inconsistent expectations, delays, and balancing thoroughness with speed. Review outcomes often include design clarification and process alignment, not just bug discovery. Implications: Organizations should treat code review as a socio-technical practice with both quality and communication functions; tooling and processes should reduce friction while preserving review's collaborative value.

23 Anonymous Author Code Review

Motivation: Knowledge of author identity may influence review outcomes through bias, reputation effects, or social dynamics, so anonymity may change review behavior. Methodology: Field experiment studying the engineering impact of anonymous author code review in practice. Key Findings: Anonymous review can alter participation, perceptions, and some aspects of reviewer behavior, but the effects are nuanced rather than universally positive or negative. Removing identity cues may reduce some biases while also changing social context and accountability. Implications: Review design choices influence both fairness and effectiveness; anonymity should be evaluated carefully as a workflow intervention rather than assumed to be automatically better.

24 Automated Debugging and Programmers

Motivation: Automated debugging tools promise to help developers find and fix faults, but it is unclear how much they truly help in realistic programming situations. Methodology: Empirical evaluation of automated debugging techniques with programmers, examining both task outcomes and human interaction with the tools. Key Findings: Automated debugging can provide useful guidance, but programmers do not always benefit as much as tool-centric evaluations suggest. Effectiveness depends on how understandable, trustworthy, and actionable the debugging output is. Tools may help locate faults, yet developers still need to reason through the explanation and integrate it into their own understanding. Implications: Debugging tools should be evaluated not just by algorithmic accuracy but by their impact on human comprehension, decision-making, and actual debugging success.